

Functional Programming Presentation Guide

Table of Contents

Guía breve de exposición: programación funcional

Objetivo de la sustentación

Diapositiva 1 — Problema real

Diapositiva 2 — Arquitectura híbrida

Diapositiva 3 — Interfaces funcionales

Diapositiva 4 — Función de orden superior

Diapositiva 5 — Composición y pureza

Diapositiva 6 — Regla de negocio demostrable

Diapositiva 7 — Pruebas

Diapositiva 8 — Valor para el cliente

Demo segura de 90 segundos

Mapa de respaldo para preguntas

Preguntas difíciles y respuestas correctas

“¿Usar `map` ya es programación funcional?”

“¿Todas las funciones son puras?”

“¿JSDoc obliga a respetar la interfaz?”

“¿Una ausencia justificada cuenta como presente?”

“¿Dónde se usa en el producto?”

Lista final antes de exponer

Guía breve de exposición: programación funcional

Objetivo de la sustentación

Demostrar en 6 a 8 minutos que American Latin Class usa programación funcional e interfaces funcionales en una necesidad real: transformar datos académicos y financieros autorizados en reportes para la dirección.

Diapositiva 1 — Problema real

Mensaje: un director necesita indicadores confiables de asistencia, recaudo, cartera, ocupación, prospectos y calidad de datos.

Qué decir:

Antes, si mezclamos consultas, permisos y fórmulas dentro del controlador, cada cálculo es más difícil de probar. Separamos los efectos externos del núcleo de cálculo.

Diapositiva 2 — Arquitectura híbrida

Controlador con I/O —datos + fecha→ módulo funcional → reporte

- `ReportsController` autentica, limita por sede, consulta y filtra el periodo.
- `reportMetrics.js` calcula sin consultar la base de datos.
- La interfaz web consume el JSON para tablas, gráficos y alertas.

Frase importante: no afirmamos que todo el sistema sea funcional; aplicamos el paradigma donde aporta claridad y capacidad de prueba.

Diapositiva 3 — Interfaces funcionales

Mostrar al inicio de `06Code/src/functional/reportMetrics.js`:

```
Predicate<T>
Mapper<T, R>
Reducer<T, R>
NumberSelector<T>
ReportMetricCalculator<TContext, TResult>
```

Qué decir:

JavaScript no tiene `@FunctionalInterface` como Java. Representamos el contrato con JSDoc y lo materializamos pasando funciones como valores. JSDoc documenta y ayuda al editor; la ejecución real ocurre mediante callbacks.

Diapositiva 4 — Función de orden superior

Mostrar:

```
const sumBy = (items, selector) => toMoney(
  items.reduce((total, item) => total + Number(selector(item) || 0), 0),
);
```

Ejemplos de uso:

```
sumBy(payments, (payment) => payment.amount);
sumBy(events, (event) => event.showIncome);
```

Idea a defender: `sumBy` recibe comportamiento; el `NumberSelector` decide qué número extraer sin duplicar el algoritmo de suma.

Diapositiva 5 — Composición y pureza

Mostrar:

```
const branches = source.branches.map((branch) =>
  createBranchReport(branch, source, referenceDate)
);
const totals = branches.reduce(combineBranchTotals, emptyTotals());
```

Qué decir:

- `map` transforma cada sede en un reporte.
- `reduce` compone los reportes en un consolidado.
- No se altera `source`.
- La fecha se inyecta; con entrada y fecha iguales, la salida es igual.

Diapositiva 6 — Regla de negocio demostrable

Usar la asistencia porque muestra que la implementación no es artificial:

```
2 asistencias físicas + 2 ausencias
1 ausencia tiene excusa aprobada
```

Tasa cruda: $2 / 4 = 50 \%$
Tasa ajustada: $2 / 3 = 66,67 \%$

Qué decir:

Justificar no convierte una ausencia en presencia. Conservamos el dato físico para auditoría y solo excluimos la excusa aprobada del denominador ajustado.

También se puede mencionar:

- ocupación y lista de espera;
- recaudo bruto, reversos y neto;
- cartera vencida por antigüedad;
- conversión de prospectos;
- alertas de integridad.

Diapositiva 7 — Pruebas

Mostrar `06Code/tests/unit/report-metrics.test.js` y ejecutar:

```
cd 06Code
npm test -- --runInBand --runTestsByPath tests/unit/report-
metrics.test.js
```

Evidencia verificada:

```
Test Suites: 1 passed, 1 total
Tests:      9 passed, 9 total
```

Se comprueban helpers funcionales, asistencia, finanzas, ocupación, embudo, alertas, consolidado e inmutabilidad.

Diapositiva 8 — Valor para el cliente

Cierre sugerido:

La programación funcional no se añadió solo para cumplir una materia. El director recibe métricas repetibles y auditables; el equipo puede probar cada fórmula sin servidor ni base de datos; y una regla nueva se agrega componiendo funciones pequeñas, sin convertir el controlador en un bloque difícil de mantener.

Demo segura de 90 segundos

1. Abrir `reportMetrics.js` y mostrar los cinco contratos JSDoc.
2. Ir a `sumBy` y señalar que recibe el selector.
3. Ir a `buildAcademicReport` y mostrar `map` + `reduce`.
4. Abrir la prueba de ausencia justificada y explicar 50 % frente a 66,67 %.
5. Ejecutar la prueba focal y mostrar `9 passed`.
6. Si el sistema está levantado y hay sesión autorizada, abrir Reportes y relacionar los totales/gráficos con el JSON calculado.

Mapa de respaldo para preguntas

Si preguntan por...	Respuesta corta	Evidencia
Interfaz funcional	Contrato de una función con una sola responsabilidad; aquí se documenta con JSDoc	<code>Predicate</code> , <code>Mapper</code> , <code>Reducer</code> , <code>NumberSelector</code> , <code>ReportMetricCalculator</code>
Función de orden superior	Recibe o devuelve otra función	<code>sumBy</code> , <code>countBy</code> , <code>idSetFrom</code>
Función pura	Mismos datos y fecha inyectada producen el mismo resultado, sin efectos externos	<code>calculateAttendanceMetrics</code> , <code>buildAcademicReport</code>
Inmutabilidad	No se modifica la fuente; se crean arreglos y objetos nuevos	prueba <code>report functions do not mutate the input source</code>
Composición	El resultado de funciones pequeñas construye el reporte mayor	<code>createBranchReport</code> dentro de <code>buildAcademicReport</code>
Efectos secundarios	Se mantienen en el controlador/repositorio	<code>ReportsController.reportSource</code>
Regla de beca/promoción	Está en servicios porque necesita datos persistentes; reutiliza métricas de asistencia finalizada	<code>tests/unit/rules.test.js</code>
Por qué no Java	El proyecto es JavaScript; JSDoc expresa el contrato equivalente, pero no se debe confundir con validación runtime	typedefs al inicio del módulo

Preguntas difíciles y respuestas correctas

“¿Usar `map` ya es programación funcional?”

No por sí solo. Aquí se combina con funciones de primera clase, ausencia de mutación, composición, separación de efectos e inyección explícita de la fecha.

“¿Todas las funciones son puras?”

No. Consultar la base de datos y responder HTTP son efectos necesarios. Se aíslan en el controlador y los servicios. El núcleo de métricas es determinista cuando recibe la fecha de referencia, como ocurre en producción y en pruebas.

“¿JSDoc obliga a respetar la interfaz?”

No en tiempo de ejecución. Describe el contrato y permite asistencia estática del editor. Las pruebas automatizadas verifican el comportamiento real.

“¿Una ausencia justificada cuenta como presente?”

No. Permanece como ausencia en la tasa cruda y, si fue aprobada, se excluye del denominador ajustado. Así el reporte es honesto y también justo.

“¿Dónde se usa en el producto?”

En `GET /api/v1/reports/general`, el resumen y detalle por sede, y en los datos que alimentan las visualizaciones del módulo Reportes.

Lista final antes de exponer

- Ejecutar la prueba focal y conservar visible el resultado.
- Abrir previamente los tres archivos: módulo, controlador y prueba.
- Usar una fecha fija al explicar determinismo.
- No repetir los conteos antiguos de 5 pruebas o 46 pruebas totales.
- No decir que `justified` equivale a asistencia física.
- No presentar `RulesService` como una función pura.